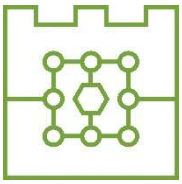


Politechnika Krakowska
im. Tadeusza Kościuszki

Wydział Informatyki i Matematyki



Wiktor Zmiendak

Numer albumu: 142706

Sprawozdanie

Projekt 5

FIPS 140-2/3

Kraków 2026

Cel zadania

Celem ćwiczenia było przygotowanie dwóch oddzielnych programów demonstrujących podstawowe operacje kryptograficzne przy użyciu dwóch różnych bibliotek dostępnych w wybranym języku programowania. W projekcie zostały wykorzystane biblioteki dostępne w środowisku Python, którymi są cryptography oraz narzędzie systemowe OpenSSL CLI. Dobór ten był uzasadniony wymaganiem, aby narzędzia wykorzystywane w zadaniu mogły być powiązane z walidacją FIPS 140-2 lub FIPS 140-3.

Implementacja

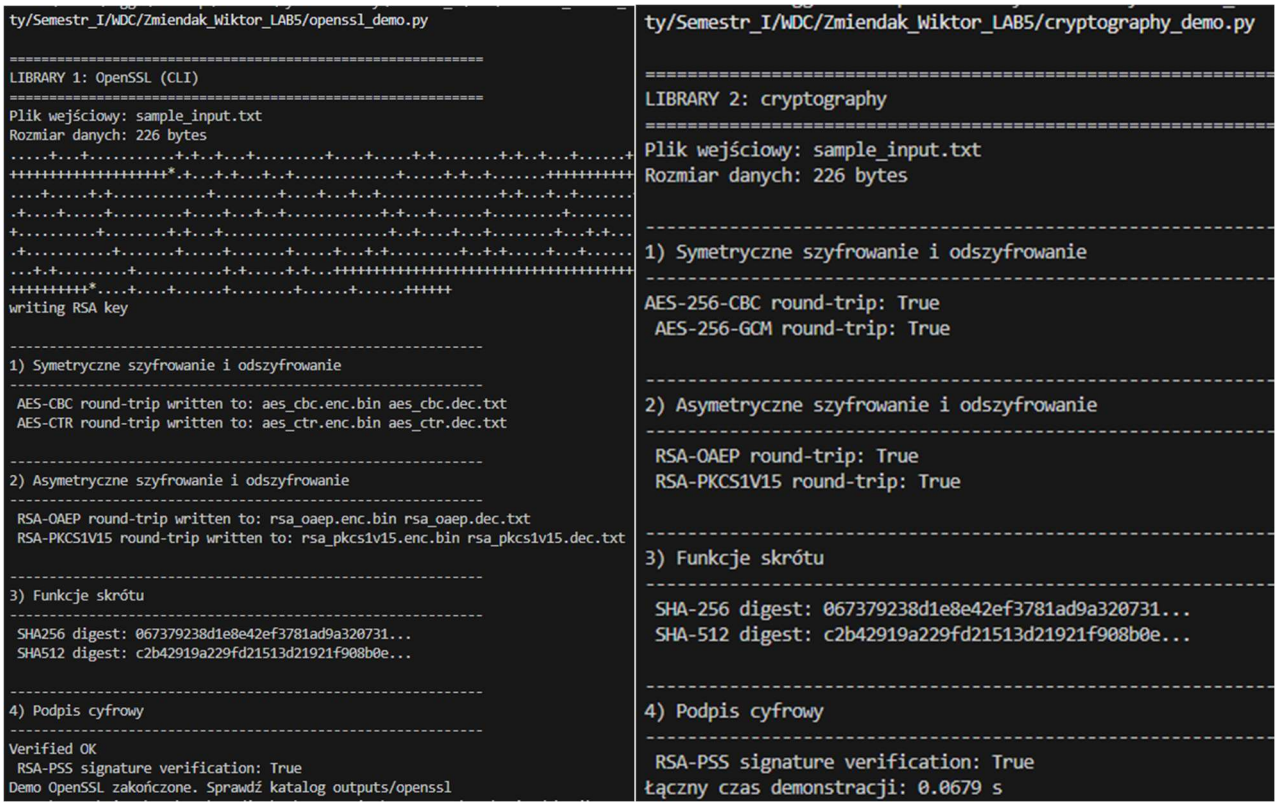
W pierwszym etapie przygotowano dwa osobne skrypty. Każdy z nich został zaprojektowany tak, aby działał na tym samym pliku wejściowym i wykonywał identyczny zestaw zadań kryptograficznych. Programy najpierw odczytują dane z pliku tekstowego, następnie wykonują szyfrowanie i odszyfrowywanie danych, obliczają skróty pliku, tworzą podpis cyfrowy oraz weryfikują jego poprawność. Na końcu wyniki są zapisywane do katalogów wyjściowych, dzięki czemu można łatwo sprawdzić poprawność działania każdej operacji.

W skrypcie opartym o cryptography zastosowano dwa algorytmy szyfrowania symetrycznego AES-256-CBC oraz AES-256-GCM. Pierwszy z nich wymagał dopełnienia danych metodą PKCS7, ponieważ AES w trybie blokowym wymaga pełnych bloków wejściowych. Tryb GCM został wykorzystany jako przykład szyfrowania z uwierzytelnieniem. W części asymetrycznej wykorzystano RSA-OAEP oraz RSA-PKCS1v15, przy czym dla większych danych zastosowano dzielenie na bloki, ponieważ RSA może szyfrować tylko ograniczoną ilość danych naraz. Następnie obliczono dwa skróty, SHA-256 i SHA-512, a na końcu utworzono podpis RSA-PSS i sprawdzono jego poprawność przy użyciu klucza publicznego.

W skrypcie wykorzystującym OpenSSL CLI zrealizowano ten sam zakres funkcji przy czym dla części symetrycznej zamiast AES-GCM użyto AES-CTR w jednym miejscu, ponieważ w niektórych buildach OpenSSL polecenie „openssl enc” może nie obsługiwać rozszerzeń AEAD. W części asymetrycznej wykonano RSA-OAEP oraz RSA-PKCS1v15, również z podziałem danych na bloki, ponieważ OpenSSL, podobnie jak biblioteka cryptography, nie pozwala zaszyfrować całego większego pliku jednym wywołaniem RSA. W części skrótów obliczono SHA-256 i SHA-512, a w części podpisu utworzono i zweryfikowano podpis RSA-PSS. Skrypt generuje także własną parę kluczy RSA, co upraszcza uruchamianie demonstracji bez ręcznego przygotowywania materiału kluczowego.

Testowanie

Testy przeprowadzono uruchamiając oba skrypty na pliku „sample_input.txt” znajdującym się w katalogu projektu. Każdy etap demonstracji w obu skryptach został sprawdzony przez porównanie odszyfrowanego wyjścia z oryginalnym plikiem wejściowym oraz przez weryfikację podpisu cyfrowego. Testowe uruchomienia w środowisku deweloperskim potwierdziły poprawność wszystkich wymienionych etapów.



Porównanie bibliotek

Pod względem łatwości użycia biblioteka cryptography dostarcza wysokopoziomowe i idiomatyczne API napisane z myślą o programistach Pythona, dzięki czemu implementacja typowych zadań kryptograficznych jest prostsza, mniej podatna na błędy i bardziej czytelna niż ręczne składanie wywołań do narzędzia CLI. W praktyce oznacza to, że cryptography wymaga mniej kodu, a testowanie oraz utrzymanie jest łatwiejsze, zwłaszcza dla osób, które nie muszą zagłębiać się w niskopoziomowe szczegóły algorytmów. OpenSSL uruchamiany jako zewnętrzne narzędzie CLI daje z kolei bardzo dużą kontrolę nad parametrami i pozwala bezpośrednio obserwować, jakie operacje są wykonywane w warstwie binariów, jednak taka ekspozycja niskiego poziomu wymaga ręcznego konstruowania poleceń, zarządzania plikami tymczasowymi i uważnego ustawiania opcji, co zwiększa złożoność implementacji i ryzyko pomyłek.

Jeżeli chodzi o czytelność dokumentacji, to w OpenSSL jest on obszerna i szczegółowa, lecz ma charakter techniczny i może wymagać większego doświadczenia, aby szybko znaleźć praktyczne przykłady wykorzystania. Dokumentacja biblioteki cryptography jest natomiast skonstruowana w sposób bardziej przyjazny dla programisty Pythona i zawiera liczne przykłady wysokiego poziomu, co przyspiesza wdrożenie i zmniejsza prawdopodobieństwo typowych błędów implementacyjnych.

Pod względem wydajności surowe implementacje algorytmów kryptograficznych zwykle opierają się na zoptymalizowanych, niskopoziomowych bibliotekach, więc przepustowość algorytmów symetrycznych i asymetrycznych jest porównywalna przy tych samych ustawieniach i tym samym kodzie maszynowym. Różnice wydajnościowe pojawiają się natomiast na poziomie integracji. Cryptography działa wewnątrz procesu Pythona i nie wymaga uruchamiania zewnętrznych procesów ani zapisu tymczasowych plików, dlatego przy wielu krótkich operacjach oraz dla danych o małej i średniej wielkości jej narzut jest zwykle mniejszy niż przy użyciu OpenSSL CLI, które wiąże się z kosztem uruchamiania procesów i operacjami wejścia/wyjścia.

Najważniejsza różnica dotyczy jednak dostępnych certyfikacji i walidacji FIPS. OpenSSL CLI jest naturalnym kandydatem do pracy w środowisku FIPS, ponieważ może być uruchomiony na binariach i modułach zwalidowanych w ramach CMVP/NIST, o ile konkretna instalacja rzeczywiście ma taki status. W przypadku cryptography sytuacja jest bardziej skomplikowana, ponieważ biblioteka sama w sobie nie jest osobno certyfikowana jako moduł FIPS, lecz korzysta z backendu OpenSSL. Oznacza to, że zgodność FIPS zależy od tego, z jakim OpenSSL została zbudowana i jak działa w danym środowisku.